Lecture #24: Distributed OLAP Databases

15-445/645 Database Systems (Fall 2020)
https://15445.courses.cs.cmu.edu/fall2020/
Carnegie Mellon University
Prof. Andy Pavlo

1 Decision Support Systems

For a read-only OLAP database, it is common to have a bifurcated environment, where there are multiple instances of OLTP databases that ingest information from the outside world which is then fed into the back-end OLAP database, sometimes called a *data warehouse*. There is an intermediate step called *ETL*, or **E**xtract, **T**ransform, and **L**oad, which combines the OLTP databases into a universal schema for the data warehouse.

Decision support systems (DSS) are applications that serve the management, operations, and planning levels of an organization to help people make decisions about future issues and problems by analyzing historical data stored in a data warehouse.

The two approaches for modeling an analytical database are *star schemas* and *snowflake schemas*.

Star Schema

Star schemas contain two types of tables: *fact tables* and *dimension tables*. The fact table contains multiple "events" that occur in the application. It will contain the minimal unique information per event, and then the rest of the attributes will be foreign key references to outer dimension tables. The dimension tables contain redundant information that is reused across multiple events. In a star schema, there can only be one dimension-level out from the fact table. Since the data can only have one level of dimension tables, it can have redundant information. Denormalized data models may incur integrity and consistency violations, so replication must be handled accordingly. Queries on star schemas will (usually) be faster than a snowflake schema because there are fewer joins. An example of a star schema is shown in Figure 1.

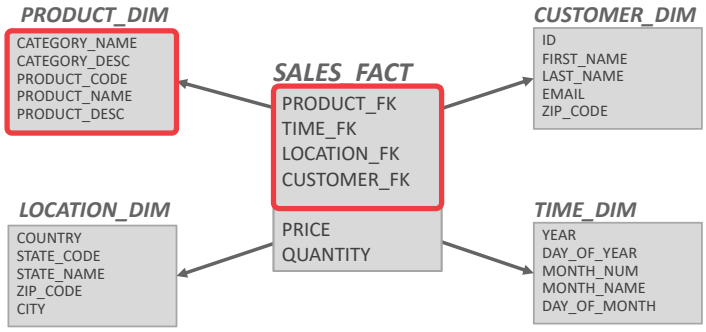

Figure 1: Star Schema – The center of the schema is the `SALES` fact table that contains key references to outer dimension tables. Because star schemas are only one-dimensional, the outer dimensional tables cannot point to other dimension tables.

Snowflake Schema

Snowflake schemas are similar to star schemas except that they allow for more than one dimension out from the fact table. They take up less storage space, but they require more joins to get the data needed for a query. For this reason, queries on star schemas are usually faster. An example of a snowflake schema is shown in Figure 2.

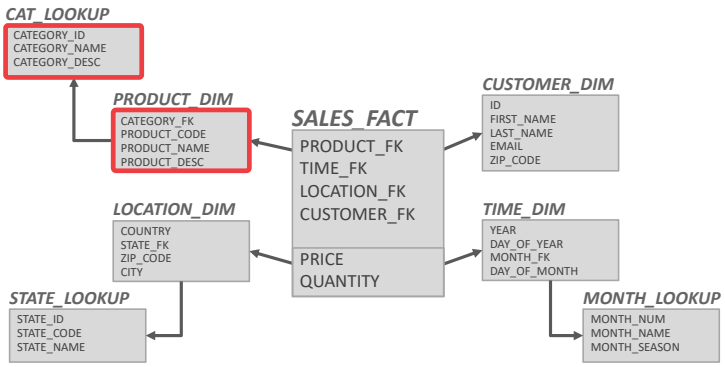

Figure 2: Snowflake Schema – The category information in the product dimension table can be broken out in the snowflake table.

2 Execution Models

A distributed DBMS's execution model specifies how it will communicate between nodes during query execution. Two approaches to executing a query are *pushing* and *pulling*.

Pushing a Query to Data

For the first approach, the DBMS sends the query (or a portion of it) to the node that contains the data. It then performs as much filtering and processing as possible where data resides before transmitting over network. The result is then sent back to where the query is being executed, which uses local data and the data sent to it, to complete the query. This is more common in a shared nothing system.

Pulling Data to Query

For the second approach, the DBMS brings the data to the node that is executing a query that needs it for processing. In other words, nodes detect which partitions of the data they can do computation on and pull from storage accordingly. Then, the local operations are propagated to one node, which does the operation on all the intermediary results. This is normally what a shared disk system would do. The problem with this is that the size of the data relative to the size of the query could be very different. A filter can also be sent to only retrieve the data needed from disk.

Query Fault Tolerance

The data that a node receives from remote sources are cached in the buffer pool. This allows the DBMS to support intermediate results that are larger than the amount of memory available. Ephemeral pages, however, are not persisted after a restart. Therefore, a distributed DBMS must consider what happens to a long-running OLAP query if a node crashes during execution.

Most shared-nothing distributed OLAP DBMSs are designed to assume that nodes do not fail during query

execution. If one node fails during query execution, then the whole query fails, which entails the entire query executing from the start. This can be expensive, as some OLAP queries can take days to execute.

The DBMS could take a snapshot of the intermediate results for a query during execution to allow it to recover if nodes fail. This operation is expensive, however, because writing data to disk is slow.

3 Query Planning

All the optimizations that we talked about before are still applicable in a distributed environment, including predicate pushdown, early projections, and optimal join orderings. Distributed query optimization is even harder because it must consider the location of data in the cluster and data movement costs.

One approach is to generate a single global query plan and then distribute *physical operators* to nodes, breaking it up into partition-specific fragments. Most systems implement this approach.

Another approach is to take the SQL query and rewrite the original query into partition-specific queries. This allows for local optimization at each node. MemSQL is an example of a system that deploys this.

4 Distributed Join Algorithms

For analytical workloads, the majority of the time is spent doing joins and reading from disk, showing the importance of this topic. The efficiency of a distributed join depends on the target tables' partitioning schemes.

One approach is to put entire tables on a single node and then perform the join. However, the DBMS loses the parallelism of a distributed DBMS, which defeats the purpose of having a distributed DBMS. This option also entails costly data transfer over the network.

To join tables R and S, the DBMS needs to get the proper tuples on the same node. Once there, it then executes the same join algorithms discussed earlier in the semester. One should always send the minimal amount needed to compute the join, sometimes entailing entire tuples.

There are four scenarios for distributed join algorithms.

Scenario 1

One of the tables is replicated at every node and the other table is partitioned across nodes. Each node joins its local data and then sends their results to a coordinating node.

Scenario 2

Both tables are partitioned on the join attribute, with IDs matching on each node. Each node performs the join on local data and then sends to a node for coalescing.

Scenario 3

Both tables are partitioned on different keys. If one of the tables is small, then the DBMS broadcasts that table to all nodes. Local joins are computed and then those joins are sent to a common node to operate the final join. This is known as a *broadcast join*.

Scenario 4

This is the worst case scenario. Both tables are not partitioned on the join key. The DBMS copies the tables by reshuffling them across nodes. Local joins are computed and then the results are sent to a common node

for the final join. If there isn't enough disk space, a failure is unavoidable. This is called a *shuffle join*.

Semi-Join

A *semi-join* is a join operator where the result only contains columns from the left table. Distributed DBMSs use semi-join to minimize the amount of data sent during joins.

It is like a natural join, except that the attributes on the right table that are not used to compute the join are restricted.

5 Cloud Systems

Vendors provide *database-as-a-service* (DBaaS) offerings that are managed DBMS environments.

Newer systems are starting to blur the lines between shared-nothing and shared-disk. For example, **Amazon Redshift** allows for simple filtering before copying data to compute nodes. There are two types of cloud systems, managed or cloud-native DBMSs.

Managed DBMSs

In a managed DBMS, no significant modification to the DBMS to be "aware" that it is running in a cloud environment. It provides a way to abstract away all the backup and recovery for the client. This approach is deployed in most vendors.

Cloud-Native DBMS

A cloud-native system is designed explicitly to run in a cloud environment. This is usually based on a shared-disk architecture. This approach is used in **Snowflake**, **Google BigQuery**, **Amazon Redshift**, and **Microsoft SQL Azure**.

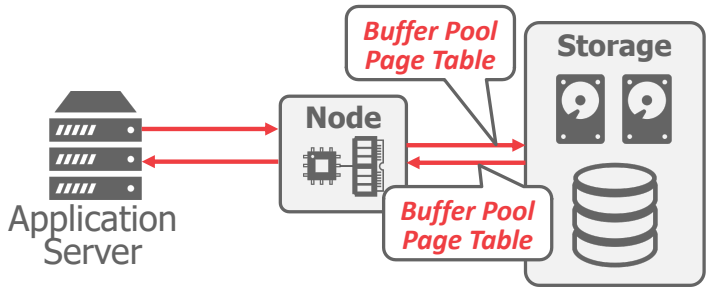

Figure 3: Serverless Database – When the application server becomes idle, the user must pay for resources in the node that are not being used. In a serverless database, when the application server stops, the DBMS takes a snapshot of pages in the buffer pool and writes it out to shared disk so that the computation can be stopped. When the application server returns, the buffer pool page table restores the previous state in the node.

Serverless Databases

Rather than always maintaining compute resources for each customer, a *serverless DBMS* evicts tenants when they become idle, checkpointing the current progress in the system to disk. Now, a user is only paying

for storage when not actively querying. A diagram of this is shown in Figure 3.

Universal Formats

Most DBMSs use a proprietary on-disk binary file format for their databases. The only way to share data between systems is to convert data into a common text-based format, including CSV, JSON, and XML. There are new open-source binary file formats, which cloud vendors and distributed database systems support, that make it easier to access data across systems. Writing a custom file format would give way to better compression and performance, but this gives way to better interoperability.

Notable examples of universal database file formats:

- **Apache Parquet:** Compressed columnar storage from Cloudera/Twitter.
- **Apache ORC:** Compressed columnar storage from **Apache Hive**.
- **Apache CarbonData:** Compressed columnar storage with indexes from Huawei.
- **Apache Iceberg:** Flexible data format that supports schema evolution from Netflix.
- **HDF5:** Multi-dimensional arrays for scientific workloads.
- **Apache Arrow:** In-memory compressed columnar storage from Pandas/Dremio.